Control de flujo y JavaScript moderno

Decisiones, bucles y funciones como valores

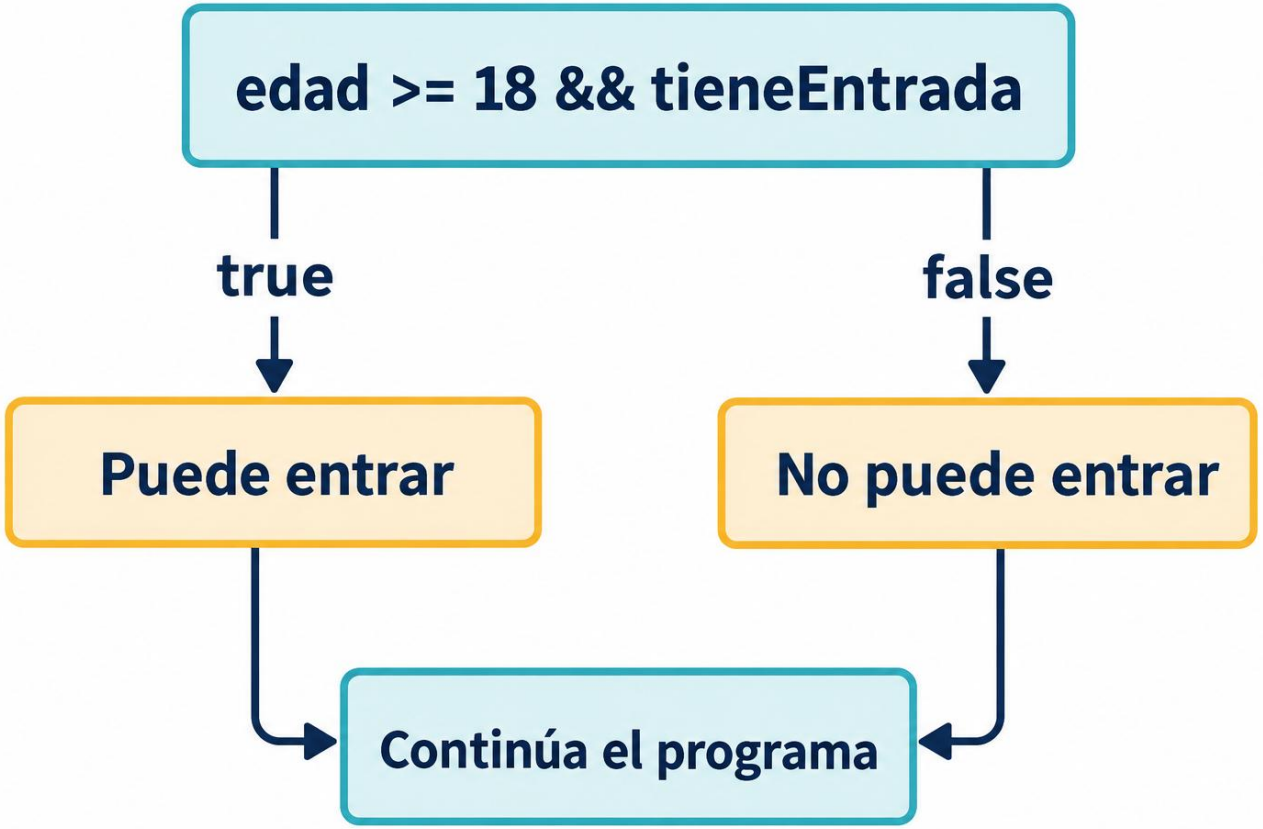
if / else: una decisión

```
const edad = 20;
const tieneEntrada = true;

if (edad >= 18 && tieneEntrada) {
  console.log("Puede entrar");
} else {
  console.log("No puede entrar");
}
```

Se ejecuta una sola rama. && exige que ambas condiciones se cumplan.

El recorrido de una condición



Con edad = 20 y tieneEntrada = true, el recorrido pasa por «Puede entrar».

else if: importa el orden

```
const nota = 7;
if (nota >= 9) {
  console.log("Sobresaliente");
} else if (nota >= 7) {
  console.log("Notable");
} else if (nota >= 5) {
  console.log("Aprobado");
} else {
  console.log("Suspenso");
}
```

La primera condición verdadera decide la rama.

¿Qué muestra con 7?
¿Y con 4 o con 9?

for: repetir con un contador

```
for (let i = 1; i <= 5; i++) {
  console.log(i);
}
```

1. Inicialización: let i = 1, una sola vez
2. Condición: i <= 5, antes de cada vuelta
3. Cuerpo: muestra i
4. Actualización: i++, después del cuerpo

Salida: 1, 2, 3, 4, 5. Al llegar a 6, la condición es falsa.

Un array: recorrido por índices

```
const nombres = ["Ana", "Luis", "Eva"];

for (let i = 0; i < nombres.length; i++) {
  console.log(i, nombres[i]);
}

// 0 Ana
// 1 Luis
// 2 Eva
```

Los índices empiezan en 0. El último índice es nombres.length - 1.

for...of: recorrido por valores

```
const nombres = ["Ana", "Luis", "Eva"];

for (const nombre of nombres) {
  console.log(nombre);
}
// Ana, Luis, Eva
```

for clásico: útil si necesitas el índice.
for...of: útil si necesitas cada valor.

for...of también recorre otros iterables, como las cadenas de texto.

Las tres partes de for son opcionales

```
for (inicializador; condición; actualizador) {
  // cuerpo
}
```

Sin inicializador: puedes preparar el estado antes.

Sin condición: no hay una comprobación que lo detenga.

Sin actualizador: puedes cambiar el estado en el cuerpo.

Puedes omitir una, dos o las tres partes. Los dos puntos y coma permanecen.

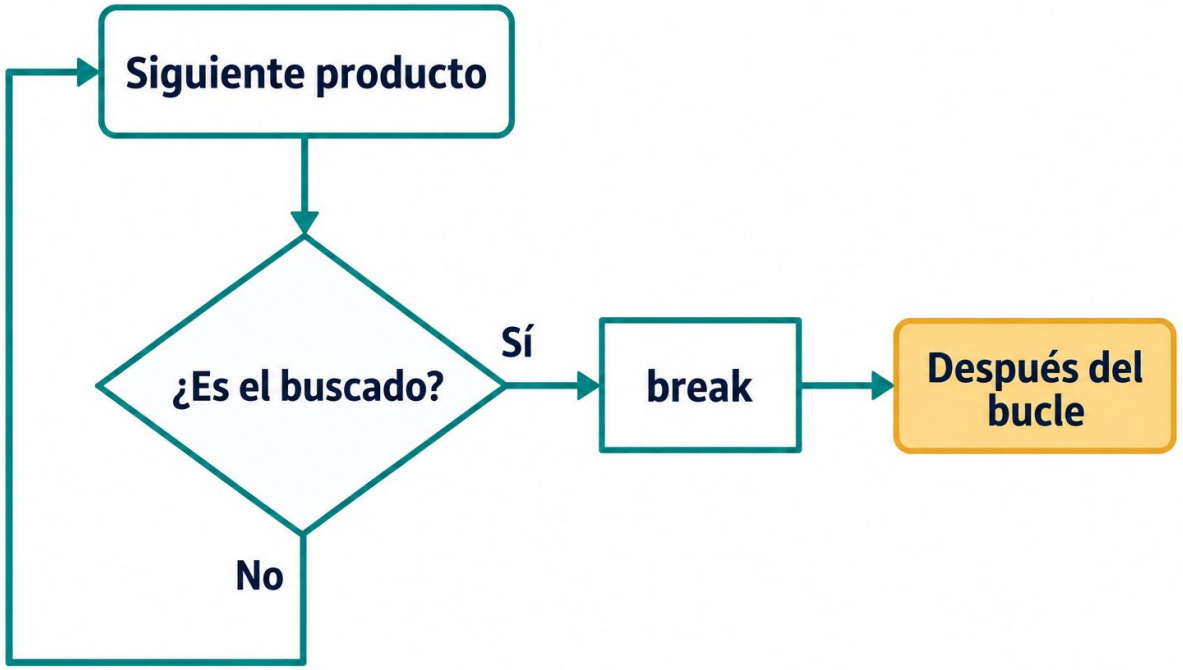
break: terminar el bucle

```
const productos = ["Teclado", "Monitor", "Ratón"];

for (const producto of productos) {
  console.log(producto);
  if (producto === "Monitor") {
    break;
  }
}
// Teclado, Monitor
```

Al encontrar Monitor, break sale del bucle. El programa continúa después.

El recorrido de break



break omite el resto de la iteración y todas las vueltas pendientes.

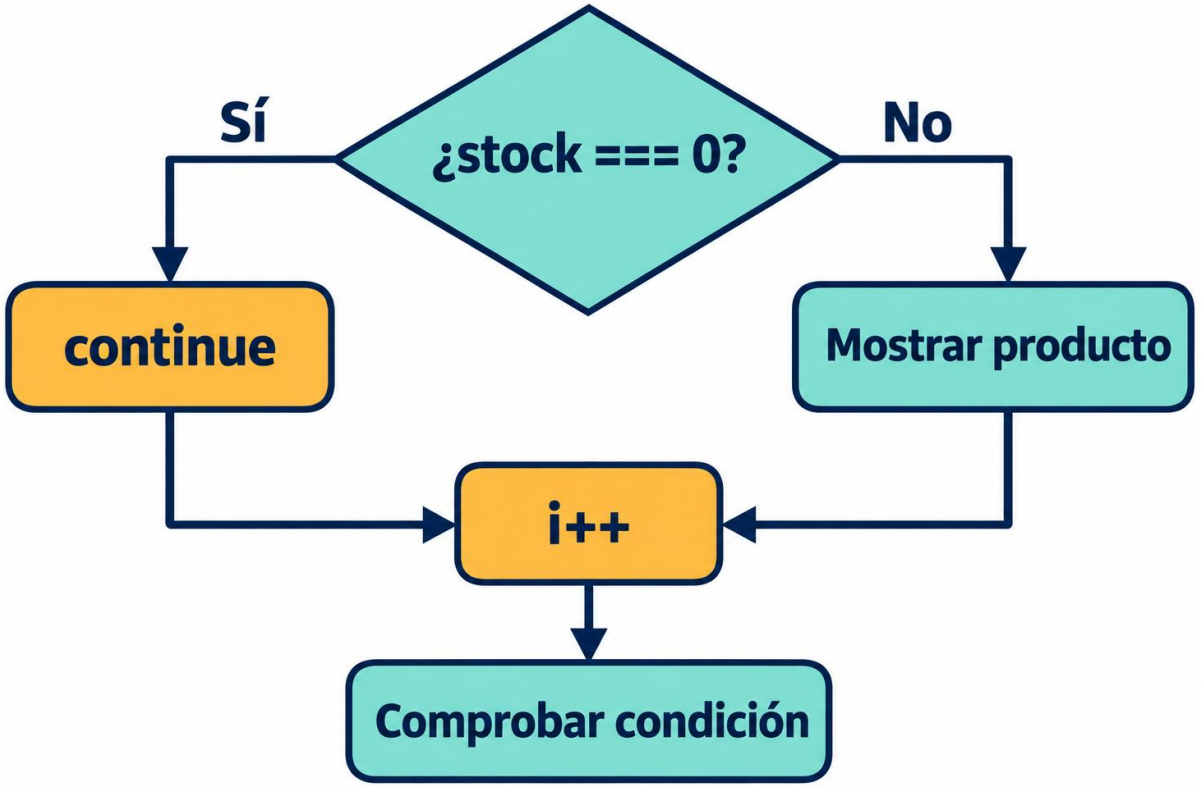
continue: saltar una iteración

```
const productos = [
  { nombre: "Teclado", stock: 0 },
  { nombre: "Monitor", stock: 3 }
];

for (let i = 0; i < productos.length; i++) {
  if (productos[i].stock === 0) continue;
  console.log(productos[i].nombre);
}
// Monitor
```

continue salta el resto del cuerpo. En este for, después ejecuta i++.

El recorrido de continue



Con stock 0 se omite Mostrar producto, pero se mantiene la actualización.

for (;;): salida con break

```
const nombres = ["Ana", "Luis", "Eva"];
let i = 0;

for (;;) {
  if (i >= nombres.length) break;
  console.log(nombres[i]);
  i++;
}
```

break termina el bucle. Sin una salida, for (;;) puede repetirse indefinidamente.

while y do...while

while

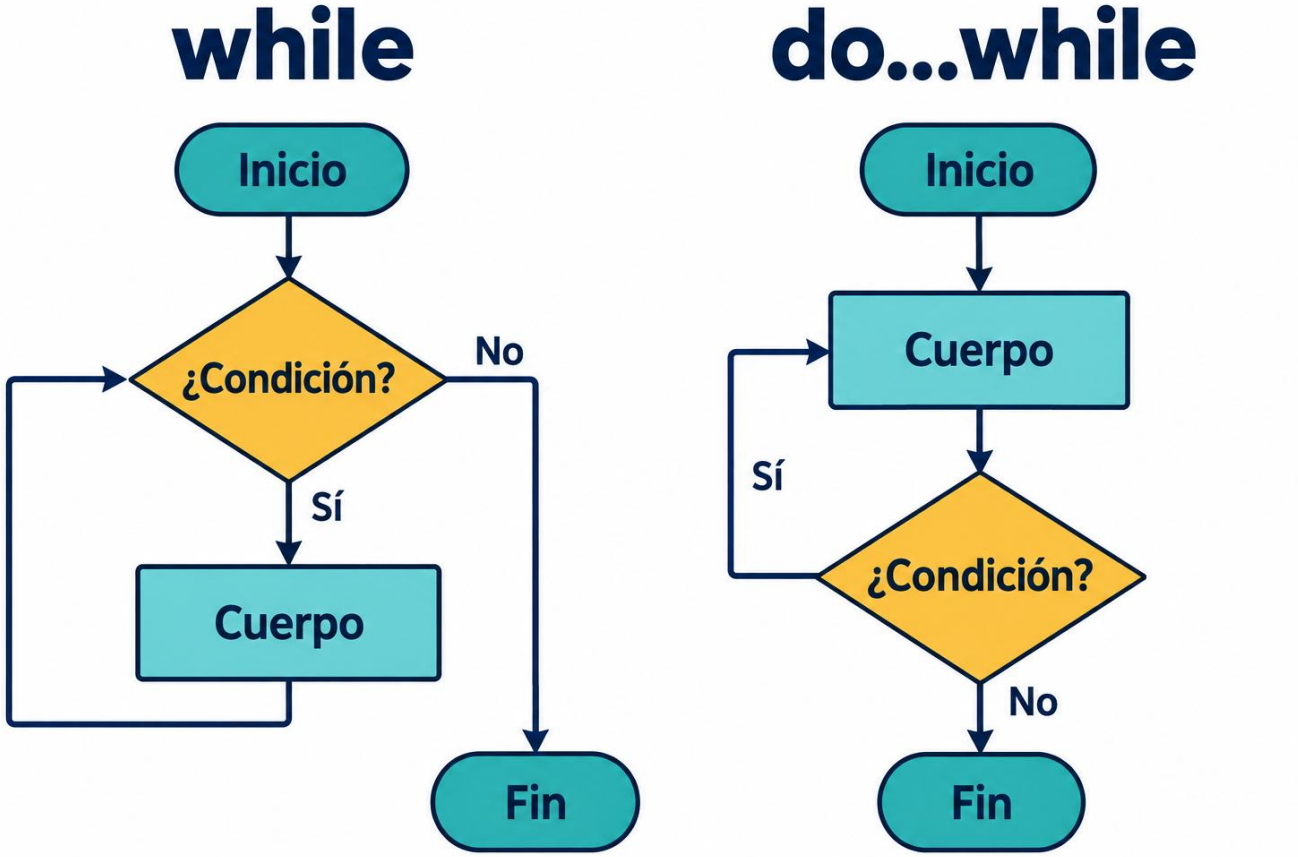
```
let i = 1;
while (i <= 5) {
  console.log(i);
  i++;
}
```

do...while

```
let i = 1;
do {
  console.log(i);
  i++;
} while (i <= 5);
```

Ambos muestran del 1 al 5. ¿Qué cambia si i empieza en 6?

La condición antes o después



do...while ejecuta el cuerpo al menos una vez.

continue en while: cuidado con el avance

```
let i = 0;

while (i < 3) {
  if (i === 2) continue;
  i++;
  console.log(i);
}
// 1, … bucle infinito
```

```
let i = 0;

while (i < 3) {
  i++;
  if (i === 2) continue;
  console.log(i);
}
// 1, 3
```

En while, continue vuelve a la condición. El avance debe ocurrir antes del salto.

null y undefined

```
const producto = {
  nombre: "Teclado",
  stock: 0,
  descuento: null
};

producto.fabricante  // undefined
producto.descuento   // null
```

undefined
No hay un valor asignado, o la propiedad no existe.

null
Un valor que expresa una ausencia intencionada.

|| y ??: dos valores de respaldo

```
producto.stock || 10       // 10
producto.stock ?? 10       // 0

producto.descuento ?? 0    // 0
```

|| usa el respaldo si el valor es falsy.
?? usa el respaldo solo si es null o undefined.

Un stock de 0 es un dato válido: ?? lo conserva.

Los valores que activan el respaldo

Con ||

```
false
0 y -0
0n
""
NaN
null
undefined
```

Con ??

```
null
undefined
```

0, false y "" se conservan.

Los operadores devuelven un valor y solo evalúan el respaldo si lo necesitan.

Predicción rápida

```
"" || "Anónimo"

"" ?? "Anónimo"

null ?? 0

false ?? true
```

¿Qué devuelve cada expresión?

¿Es el primer valor un dato que queremos conservar?

Ternario: elegir un valor

```
const stock = 3;
let estado;
if (stock > 0) {
  estado = "Disponible";
} else {
  estado = "Agotado";
}
```

La condición decide qué valor asignamos a estado.

Resultado:
"Disponible"

La misma elección con una expresión

```
const estado = stock > 0
  ? "Disponible"
  : "Agotado";
```

condición ? valor si true : valor si false

Útil para obtener un valor. Para bloques de acciones, if/else suele ser más claro.

Una función también es un valor

```
function duplicar(numero) {
  return numero * 2;
}

const operacion = duplicar;

console.log(operacion(5)); // 10
```

operacion guarda la misma función que duplicar. Todavía no la ejecuta.

Guardar una función o ejecutarla

Referencia a la función

```
const operacion = duplicar;
```

Resultado de ejecutar la función

```
const resultado = duplicar(5); // 10
```

Los paréntesis de llamada ejecutan la función. return devuelve el resultado.

Una función como argumento

```
function ejecutar(operacion, valor) {
  return operacion(valor);
}

console.log(ejecutar(duplicar, 5));
// 10
```

Un callback es una función que pasamos a otra función para que esta la invoque.

El recorrido del callback



ejecutar recibe duplicar y la llama con el valor 5.

Arrow functions: otra sintaxis

```
function duplicar(numero) {
  return numero * 2;
}
```

La misma operación, escrita como arrow function

```
const duplicar = (numero) => {
  return numero * 2;
};
```

Retorno implícito

```
const duplicar = numero => numero * 2;
```

Un parámetro: los paréntesis son opcionales.
Una expresión sin llaves: su valor se devuelve automáticamente.

Con llaves, hace falta return para devolver ese resultado.

Un callback escrito en la llamada

```
ejecutar(numero => numero * 2, 5);
// 10
```

Primer argumento: una función que duplica.
Segundo argumento: el valor 5.

Si necesitas reutilizar la función, puedes guardarla en una variable.

Mini-reto: el producto

```
const producto = {
  nombre: "Monitor",
  precio: 200,
  stock: 3,
  descuento: null
};
```

Crea las siguientes funciones:

1. obtenerEstado(producto)
   Arrow con ternario: "Disponible" o "Agotado".

2. obtenerPrecioFinal(producto)
   Descuento ausente: 0 %. Devuelve el precio final.

3. mostrarResultado(producto, calcularPrecio)
   Llama al callback y muestra el nombre y el precio.

Convención del ejercicio

El descuento es un porcentaje.

null o undefined significan 0 %.

Objetivo: combinar ??, ternario, arrow functions y un callback.

Comprobaciones: null da 200 €, 10 da 180 € y stock = 0 da «Agotado».